Homework 3 – Rays & Files

Due: October 17th (11:59pm)

Overview:

This homework aims to get you up and running with the basic components needed to write a successful ray tracer. We are providing you with skeleton code that shoots rays through an image plane and lights up the corresponding pixel if the ray intersects with a sphere. Historically, this is the part of a ray tracer that students struggled most with, so study the provided code very carefully.

The main programming component involves writing a file parser that reads a specification file (given in the command line), and sets the appropriate variables based on the values in that file. In the process, you'll further develop your low-level C++ skills relating to file I/O, string manipulation, and data parsing.

Getting Started:

You should use the skeleton code on the course webpage as a starting point for the assignment. There are several files that are provided for you to set up the framework and read/write image files, but for the HW, you should only change the file `parse_pga.h` or `parse_vec3.h` depending on if you are planning on using the 3D PGA library we provided, of if you would prefer to work directly with a basic 3-vector. Your choice here will have only a small effect on the HW3 parsing code, but the eventual project will be somewhat different with PGA vs Vec3.

In addition to the above file, we provide code implemented a 3D PGA library (`multivector.h` and `PGA_3D.h`), a vector library (`vec3.h`), and reading/writing images (`image_lib.h` which uses the open source *stb_image* library files: `stb_imag.h` and `stb_image_write.h`). There is no need to delve into the details of how the *stb_image* library works, but you should read the rest of the files carefully to understand how they all work, especially core files `rayTrace_pga.cpp` and `rayTrace_vec3.cpp`.

Completing this homework will directly prepare you to finish Projects 3a and 3b.

How the Program Works:

The idea behind the code interface is that all of the aspects of the 3D scene, camera, materials, etc. are specified in a single file. Similar to Project 2, the file is given to the program via the command line:

```
./raytrace ../scenes/cool-scene-file.txt
```

The file should specify all of the elements of the resulting image and scene, via a series of one-line commands. For this HW, you need to support the following 7 commands:

`sphere: x y z r`	Create a sphere with radius `r`, centered at location `(x, y, z)`. If no value is provided this defaults to (0,0,2), with radius 1.
`image_resolution: w h`	Set the image to output at a resolution of `w` pixels wide by `h` pixels in height. If no value is provided this defaults to a width of 800 and a height of 600.
`output_image: name.ext`	Set the name the image will be output as. *(The provided image library, infers the type of the image based on the file extention `.ext`).* If no value is provided this defaults to "raytraced.png"
`camera_pos: x y z`	Set the virtual camera's location to the 3D coordinate `(x, y, z)`. If no value is provided this defaults to (0,0,0).
`camera_fwd: fx fy fz`	Set the forward direction [dir. towards the virtual camera] to be the 3D direction `(fx, fy, fz)`. If no value is provided this defaults to (0,0,-1).
`camera_up: ux uy uz`	Set the up-alignment of the virtual camera to be the 3D direction `(ux, uy, uz)`. If no value is provided this defaults to (0,1,0).
`camera_fov_ha: ha`	Set the half-angle of the vertical field-of-view of the virtual camera to be `ha` degrees. *(The horizontal fov can be determined via the image aspect ratio.)* If no value is provided this defaults to 35 degrees.

Additionally, you must skip any line that begins with `#` as this indicates a comment.

Lastly, the file is not required to provide an orthogonal camera basis; rather, it is the responsibility of the parser to infer a set of orthogonal camera vectors given the provided up and forward directions.

What You Have To Do:

Parsing commands in the file

You must update the `parseSceneFile()` function to support the 7 commands given above, and to ignore comments. When the function is completed, all the of the global variables should be set as specified by the commands. If no command is given, the corresponding variables should be set their default values. The commands can be given in any order in the file, and if multiple commands are given the most recent command should override any previous commands.

Providing an orthogonal camera basis

After parsing the file, you must also complete the following steps in the `parseSceneFile()` function:

1. Infer the camera's *right*-ward direction based on the provided *up* and *forward* directions, using the right-hand rule.
2. Normalize all three camera vectors, *up*, *right*, and *forward* to ensure they are unit magnitude.
3. Ensure all three camera vectors, *up*, *right*, and *forward* are orthogonal to each other. If not, you must orthogonalize the vectors.

The resulting camera vectors: *up*, *right*, and *forward* should form an orthogonal basis. The subsequent ray tracing code can then safely assume an orthogonal basis for the camera.

HW3 Online Quiz

In addition to the coding component described above, there is a quiz posted to the course webpage with a few ray-tracing related problems to work out by-hand.

The quiz is at this link: https://canvas.umn.edu/courses/518714/quizzes/1117836
The quiz password is: RAYS

Submission Details:

You will turn in only the file *parse_pga.h* or *parse_vec3.h*, depending on what version you choose to work with. Additionally, your code must be able compile using the following command on the lab machines:

```
g++ -fsanitize=address -std=c++14 rayTrace_pga.cpp -o ray
```
or
```
g++ -fsanitize=address -std=c++14 rayTrace_vec3.cpp -o ray
```

There are a few example files provided for comparison and with the resulting renderings. You should also run your own tests.